

Graphs: Shortest Path Algorithms

A Problem-Solving Companion to Striver's A2Z and SDE Sheets

Maruri Saikiran

June 30, 2026

Contents

1	Graphs: Shortest Path Algorithms	1
1.1	Relaxation: The One Operation Every Algorithm Here Repeats	1
1.2	Shortest Path in an Undirected Graph with Unit Weights	2
1.3	Dijkstra's Algorithm	3
1.4	Number of Ways to Arrive at Destination	5
1.5	Path With Minimum Effort	7
1.6	Cheapest Flights Within K Stops	9
1.7	Bellman-Ford Algorithm	10
1.8	Floyd-Warshall Algorithm	12
1.9	City With the Smallest Number of Neighbors at a Threshold Distance	14

Chapter 1

Graphs: Shortest Path Algorithms

Plain BFS already computes shortest paths – but only when every edge costs exactly the same. The moment edges carry different weights, BFS’s level-by-level guarantee breaks, and a new idea is needed: always expand whichever frontier node currently has the smallest known distance. That single idea, plus two ways of handling weights BFS and that idea still can’t manage (negative edges, and all pairs at once), is this entire chapter.

1.1 Relaxation: The One Operation Every Algorithm Here Repeats

Every algorithm in this chapter, however different its outer loop, performs the same inner operation, called **relaxation**: for an edge $u \rightarrow v$ with weight w , if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v] \leftarrow \text{dist}[u] + w$ (a shorter path to v was just found, via u). The algorithms differ only in **which edges they relax, and in what order**.

What This Chapter Covers, and How Each Piece Differs

- **Shortest Path, Unit Weights (Section 1.2)**: plain BFS already *is* correct relaxation when every edge weighs 1 – level order and shortest distance coincide exactly.
- **Dijkstra’s Algorithm (Section 1.3)**: for non-negative weights, always relax edges out of whichever unfinished node currently has the **smallest known** distance, using a priority queue to find that node quickly.
- **Number of Ways to Arrive (Section 1.4)** and **Path With Minimum Effort (Section 1.5)**: Dijkstra’s frontier-expansion order, reused with the relaxation rule itself changed – counting ties instead of only keeping the minimum, and minimizing the *maximum* edge on a path instead of the sum.
- **Cheapest Flights Within K Stops (Section 1.6)**: relaxation bounded to a fixed number of rounds, since Dijkstra’s “always pick the smallest” greedy choice can skip a path that is cheaper only because it uses more stops.
- **Bellman-Ford (Section 1.7)**: relax **every** edge, $V-1$ times over, which is what makes it correct even with **negative** edge weights (where Dijkstra’s greedy frontier choice can be wrong) – and a V -th round of relaxation that still finds an improvement reveals a negative cycle.
- **Floyd-Warshall (Section 1.8)** and **City with Smallest Number of Neighbors (Section 1.9)**: relax every pair of nodes through every possible intermediate node, computing **all-pairs** shortest distances at once.

1.2 Shortest Path in an Undirected Graph with Unit Weights

Problem Statement. Given an undirected graph where every edge has weight 1, and a source node, find the shortest distance from the source to every other node.

Pattern Recognition

“Shortest path, every edge the same cost” is exactly what plain BFS already computes: BFS processes nodes in strictly non-decreasing order of distance from the source, so the level at which a node is first reached **is** its shortest distance. No relaxation, priority queue, or weight comparison is needed at all.

Why unit weights make BFS sufficient

The general shortest-path relaxation rule, $dist[v] = \min(dist[v], dist[u] + w)$, collapses to $dist[v] = dist[u] + 1$ when every $w = 1$ – and BFS’s queue already guarantees every node is dequeued with its *final*, minimal distance already correctly assigned (this is the same property exploited by multi-source BFS in the Graph Basics chapter’s 01 Matrix problem, here with a single source instead of many).

Algorithm

1. Initialize $dist[src] = 0$ and every other $dist[i] = \infty$.
2. BFS from the source: for each neighbor of the current node with $dist[neighbor] > dist[node] + 1$, update it and push the neighbor.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> shortestPathUnitWeights(int n, vector<vector<int>>& adj, int src) {
5      vector<int> dist(n, INT_MAX);
6      dist[src] = 0;
7      queue<int> q;
8      q.push(src);
9
10     while (!q.empty()) {
11         int node = q.front();
12         q.pop();
13
14         for (int neighbor : adj[node]) {
15             if (dist[node] + 1 < dist[neighbor]) {
16                 dist[neighbor] = dist[node] + 1;
17                 q.push(neighbor);
18             }
19         }
20     }
21     return dist;
22 }
23
24 int main() {
25     int n = 5;
26     vector<vector<int>> adj(n);
27     auto addEdge = [&](int u, int v) { adj[u].push_back(v); adj[v].push_back(u); };
28     addEdge(0, 1); addEdge(0, 2); addEdge(1, 3); addEdge(2, 3); addEdge(3, 4);

```

```

29
30     vector<int> dist = shortestPathUnitWeights(n, adj, 0);
31     for (int d : dist) cout << d << " ";
32     cout << endl; // 0 1 1 2 3
33     return 0;
34 }

```

Dry run

Setup. Take the graph above, source 0.

$dist = [0, \infty, \infty, \infty, \infty]$. Pop 0: relax 1 ($\infty \rightarrow 1$) and 2 ($\infty \rightarrow 1$); push both. Pop 1: relax 3 ($\infty \rightarrow 2$); push. Pop 2: neighbor 3 already at distance 2, and $dist[2] + 1 = 2$ is not strictly smaller, no update. Pop 3: relax 4 ($\infty \rightarrow 3$); push. Pop 4: no unrelaxed neighbors. Final: $[0, 1, 1, 2, 3]$.

Complexity

Time Complexity. $O(V + E)$. **Space Complexity.** $O(V)$ for $dist$ and the queue.

Takeaway

This problem is included specifically as a baseline: every other algorithm in this chapter is, at its core, solving the same “relax edges until distances stabilize” problem that unit-weight BFS solves trivially – they exist only because weighted, negative, or all-pairs distances break BFS’s clean level-by-level guarantee in different specific ways.

1.3 Dijkstra’s Algorithm

Problem Statement. Given a weighted graph with **non-negative** edge weights and a source node, find the shortest distance from the source to every other node.

Pattern Recognition

“Shortest path, non-negative weights” is solved by always expanding the **unfinished** node with the currently **smallest known** distance – a greedy choice, made efficient by storing candidate ($distance, node$) pairs in a **min-heap** (priority queue) so the smallest is always available in $O(\log V)$.

Why the greedy choice is correct (and needs non-negative weights)

When the node u with the smallest tentative distance is popped, that distance is guaranteed **final** – no future relaxation through any other, not-yet-finalized node x could ever produce a shorter path to u , because every such path would have to pass through some node with tentative distance $\geq dist[u]$ and then add a non-negative edge weight, which can only make the total $\geq dist[u]$. This argument breaks immediately if a negative edge weight is allowed: a long detour through a very negative edge could undercut the greedily-chosen shortest path, which is exactly why Dijkstra’s algorithm requires non-negative weights and Bellman-Ford (Section 1.7) exists for the negative case.

Algorithm

1. Initialize $dist[src] = 0$, all others ∞ . Push $(0, src)$ into a min-heap.
2. While the heap is non-empty: pop the smallest $(d, node)$; if $d > dist[node]$, this is a stale entry (a better path was already found), skip it.
3. Otherwise, relax every edge out of $node$: if $dist[node] + w < dist[neighbor]$, update and push $(dist[neighbor], neighbor)$.

Why Stale Entries Are Allowed Rather Than Removed

A node may be pushed onto the heap multiple times, once per improvement to its tentative distance, rather than removing and re-inserting it (which most heap implementations don't support efficiently). The $d > dist[node]$ check simply ignores outdated copies when they're eventually popped – correct, and simpler than maintaining a decrease-key operation.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> dijkstra(int n, vector<vector<pair<int,int>>>& adj, int src) {
5      vector<int> dist(n, INT_MAX);
6      dist[src] = 0;
7      priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
8      pq.push({0, src});
9
10     while (!pq.empty()) {
11         auto [d, node] = pq.top();
12         pq.pop();
13         if (d > dist[node]) continue; // stale entry
14
15         for (auto& [neighbor, weight] : adj[node]) {
16             if (dist[node] + weight < dist[neighbor]) {
17                 dist[neighbor] = dist[node] + weight;
18                 pq.push({dist[neighbor], neighbor});
19             }
20         }
21     }
22     return dist;
23 }
24
25 int main() {
26     int n = 5;
27     vector<vector<pair<int,int>>> adj(n); // {neighbor, weight}
28     auto addEdge = [&](int u, int v, int w) {
29         adj[u].push_back({v, w});
30         adj[v].push_back({u, w});
31     };
32     addEdge(0, 1, 4); addEdge(0, 2, 1); addEdge(2, 1, 2);
33     addEdge(1, 3, 1); addEdge(2, 3, 5); addEdge(3, 4, 3);
34
35     vector<int> dist = dijkstra(n, adj, 0);
36     for (int d : dist) cout << d << " ";
37     cout << endl; // 0 3 1 4 7
38     return 0;
39 }

```


Dry run

Setup. Take the graph above, source 0.

Heap: $[(0, 0)]$. Pop $(0, 0)$: relax 1 ($\infty \rightarrow 4$), 2 ($\infty \rightarrow 1$); push both. Heap: $[(1, 2), (4, 1)]$. Pop $(1, 2)$ (smallest): relax 1 via 2 ($4 \rightarrow 1 + 2 = 3$, improved); relax 3 via 2 ($\infty \rightarrow 1 + 5 = 6$); push $(3, 1)$ and $(6, 3)$. Heap: $[(3, 1), (4, 1)_{\text{stale}}, (6, 3)]$. Pop $(3, 1)$: relax 3 via 1 ($6 \rightarrow 3 + 1 = 4$, improved); push $(4, 3)$. Pop $(4, 1)_{\text{stale}}$: $4 > \text{dist}[1] = 3$, skip. Pop $(4, 3)$: relax 4 via 3 ($\infty \rightarrow 4 + 3 = 7$); push. Pop $(6, 3)_{\text{stale}}$: $6 > \text{dist}[3] = 4$, skip. Pop $(7, 4)$: no further relaxation. Final: $\text{dist} = [0, 3, 1, 4, 7]$.

Complexity

Time Complexity. $O((V + E) \log V)$ – each edge can trigger one heap push, and each heap operation costs $O(\log V)$. **Space Complexity.** $O(V)$ for dist , plus $O(E)$ for the heap in the worst case (allowing stale duplicate entries).

Takeaway

Dijkstra’s algorithm is BFS’s natural generalization to weighted graphs: BFS implicitly always expands the nearest unvisited node because its queue processes things in arrival order, which only equals distance order when every edge costs the same; Dijkstra’s makes that “always expand the nearest” rule explicit via a priority queue, restoring correctness for arbitrary non-negative weights.

1.4 Number of Ways to Arrive at Destination

Problem Statement. Given n cities and a list of bidirectional roads with travel times, count the number of **distinct shortest-time** ways to travel from city 0 to city $n-1$, modulo $10^9 + 7$.

Pattern Recognition

“Count the number of ways to achieve the shortest distance” pairs Section 1.3’s dist array with a parallel ways array, updated by the identical reset-on-improvement, accumulate-on-tie rule the DP on LIS chapter’s Number of LIS used for counting longest chains – here applied to Dijkstra’s relaxation instead of an array-indexed DP recurrence.

The counting rule, adapted to relaxation

When relaxing edge $u \rightarrow v$ with weight w , alongside the usual distance check:

- If $\text{dist}[u] + w < \text{dist}[v]$: a **strictly shorter** path to v was just found through u . Update $\text{dist}[v]$, and **reset** $\text{ways}[v] \leftarrow \text{ways}[u]$ (every previously counted way to v used a now-suboptimal distance).
- If $\text{dist}[u] + w = \text{dist}[v]$: this is **another equally short** path to v . **Accumulate:** $\text{ways}[v] += \text{ways}[u]$.

C++ implementation

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 const int MOD = 1e9 + 7;
```

```

4
5 int countPaths(int n, vector<vector<int>>& roads) {
6     vector<vector<pair<long long, long long>>> adj(n); // {neighbor, weight}
7     for (auto& r : roads) {
8         adj[r[0]].push_back({r[1], r[2]});
9         adj[r[1]].push_back({r[0], r[2]});
10    }
11
12    vector<long long> dist(n, LLONG_MAX), ways(n, 0);
13    dist[0] = 0;
14    ways[0] = 1;
15    priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<>> pq;
16    pq.push({0, 0});
17
18    while (!pq.empty()) {
19        auto [d, node] = pq.top();
20        pq.pop();
21        if (d > dist[node]) continue;
22
23        for (auto& [neighbor, weight] : adj[node]) {
24            if (dist[node] + weight < dist[neighbor]) {
25                dist[neighbor] = dist[node] + weight;
26                ways[neighbor] = ways[node];
27                pq.push({dist[neighbor], neighbor});
28            } else if (dist[node] + weight == dist[neighbor]) {
29                ways[neighbor] = (ways[neighbor] + ways[node]) % MOD;
30            }
31        }
32    }
33    return (int)(ways[n - 1] % MOD);
34 }
35
36 int main() {
37     vector<vector<int>> roads = {{0,6,7},{0,1,2},{1,2,3},{1,3,3},
38                                {6,3,3},{3,5,1},{6,5,1},{2,5,1},{0,4,5},{4,6,2}};
39     cout << countPaths(7, roads) << endl; // 4
40     return 0;
41 }

```

Dry run

Setup. Take a small graph: 0–1 (weight 1), 0–2 (weight 1), 1–3 (weight 1), 2–3 (weight 1) – two equally short paths from 0 to 3.

$dist = [0, \infty, \infty, \infty]$, $ways = [1, 0, 0, 0]$. Pop 0: relax 1 ($\infty \rightarrow 1$, reset $ways[1] = 1$); relax 2 ($\infty \rightarrow 1$, reset $ways[2] = 1$). Pop 1 (dist 1): relax 3 ($\infty \rightarrow 2$, reset $ways[3] = ways[1] = 1$). Pop 2 (dist 1): relax 3 – $dist[2] + 1 = 2 = dist[3]$, **tie**, accumulate $ways[3] += ways[2] = 1$, giving $ways[3] = 2$. Pop 3: no further relaxation. Final: 2 shortest paths to node 3 (via 1, and via 2), correctly counted.

Complexity

Time Complexity. $O((V+E) \log V)$, identical to plain Dijkstra – the *ways* bookkeeping adds only $O(1)$ work per relaxation. **Space Complexity.** $O(V + E)$ for the adjacency list, plus $O(V)$ for *dist* and *ways*.

Takeaway

The “exists / count / optimize” framing introduced in the DP on Subsequences chapter, and reused for counting longest chains in the DP on LIS chapter, extends naturally beyond dynamic programming tables – any algorithm computing an optimum via relaxation or comparison (here, Dijkstra’s distance updates) can be paired with a parallel counting array using the exact same reset-on-improvement, accumulate-on-tie rule.

1.5 Path With Minimum Effort

Problem Statement. Given a 2D grid of heights, find a path from the top-left cell to the bottom-right cell (moving 4-directionally) that minimizes the **effort** of the path, where a path’s effort is defined as the **maximum** absolute height difference between any two consecutive cells along it.

Pattern Recognition

“Minimize the maximum edge weight along a path” (a **minimax path**) is solved with the exact same frontier-expansion machinery as Section 1.3’s Dijkstra, with one change: the quantity tracked and relaxed is no longer a **sum** of edge weights, but the **running maximum** edge weight seen so far along the path.

Adapting the relaxation rule

Let $effort[cell]$ be the minimum possible “maximum height difference” achievable on some path from the start to $cell$. Moving from u to a neighbor v , the effort of extending that path is $\max(effort[u], |height[v] - height[u]|)$ – not $effort[u] + |height[v] - height[u]|$, since a path’s effort is its single worst step, not an accumulated total. The relaxation check becomes: if this candidate effort is **smaller** than the currently known $effort[v]$, update and push.

Why Dijkstra’s greedy-frontier property still applies

The same correctness argument from Section 1.3 carries over unchanged: popping the cell with the smallest known effort from the priority queue guarantees that effort is final, because every alternative path to it would have to pass through a cell with effort $\geq$ the popped value, and taking the max with any additional step can only keep the running effort the same or increase it – never decrease it. This “can only stay the same or grow” property is the minimax analogue of Dijkstra’s “non-negative weights only add” requirement.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int minimumEffortPath(vector<vector<int>>& heights) {
5     int rows = heights.size(), cols = heights[0].size();
6     vector<vector<int>> effort(rows, vector<int>(cols, INT_MAX));
7     effort[0][0] = 0;
8
9     priority_queue<tuple<int,int,int>, vector<tuple<int,int,int>>, greater<>> pq;
10    pq.push({0, 0, 0}); // {effort, row, col}
11

```

```

12  int dr[] = {-1, 1, 0, 0};
13  int dc[] = {0, 0, -1, 1};
14
15  while (!pq.empty()) {
16      auto [e, r, c] = pq.top();
17      pq.pop();
18      if (e > effort[r][c]) continue;
19      if (r == rows - 1 && c == cols - 1) return e;
20
21      for (int dir = 0; dir < 4; dir++) {
22          int nr = r + dr[dir], nc = c + dc[dir];
23          if (nr >= 0 && nr < rows && nc >= 0 && nc < cols) {
24              int candidate = max(e, abs(heights[nr][nc] - heights[r][c]));
25              if (candidate < effort[nr][nc]) {
26                  effort[nr][nc] = candidate;
27                  pq.push({candidate, nr, nc});
28              }
29          }
30      }
31  }
32  return 0;
33 }
34
35 int main() {
36     vector<vector<int>> heights = {
37         {1, 2, 2},
38         {3, 8, 2},
39         {5, 3, 5}
40     };
41     cout << minimumEffortPath(heights) << endl; // 2
42     return 0;
43 }

```

Dry run

Setup. Take the 3×3 heights grid above.

The path $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2) \rightarrow (2,2)$ has consecutive differences $|2-1| = 1$, $|2-2| = 0$, $|2-2| = 0$, $|5-2| = 3$ – effort 3. A better path, $(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (2,1) \rightarrow (2,2)$, has differences $|3-1| = 2$, $|5-3| = 2$, $|3-5| = 2$, $|5-3| = 2$ – effort 2, worse on individual steps but capped lower overall. Dijkstra’s frontier expansion explores cells in increasing order of their best known effort, popping $(2,2)$ (the destination) only once its true minimum effort, 2, has been finalized.

Complexity

Time Complexity. $O(\text{rows} \cdot \text{cols} \cdot \log(\text{rows} \cdot \text{cols}))$, identical structure to Dijkstra over a grid graph with 4 edges per node. **Space Complexity.** $O(\text{rows} \cdot \text{cols})$ for the *effort* grid and the priority queue.

Takeaway

Dijkstra’s algorithm generalizes to any relaxation rule that is **monotonic** – extending a path by one more edge can only keep the tracked quantity the same or make it worse, never better. Sum of weights and running maximum both satisfy this, which is why the exact same priority-queue frontier expansion solves both the standard shortest-path problem and this minimax variant.

1.6 Cheapest Flights Within K Stops

Problem Statement. Given n cities, a list of flights $[from, to, price]$, a source, a destination, and an integer k , find the cheapest price to travel from source to destination using **at most k stops** (i.e. at most $k+1$ edges). Return -1 if no such route exists.

Pattern Recognition

“Shortest path, but bounded by the number of edges used” specifically **breaks** Dijkstra’s greedy frontier-expansion property: Dijkstra always finalizes whichever node has the globally smallest distance, with no regard for how many edges that path used – so it can permanently lock in a cheap-but-too-many-stops path, missing a more-expensive-but-within-budget alternative. The fix: relax **every** edge, for exactly $k+1$ **rounds**, copying distances between rounds so each round adds at most one new edge to every path – this is a bounded version of Bellman-Ford’s relaxation strategy (Section 1.7).

Why the copy between rounds is essential

If edges were relaxed directly into the live *dist* array within a single round, a single round could chain multiple relaxations together (relax $u \rightarrow v$, then immediately use the updated $dist[v]$ to relax $v \rightarrow w$ in the *same* round) – silently allowing paths with more edges than the round count intends. Using a fresh copy (*temp*) as the round’s write target, while reading from the previous round’s frozen *dist*, guarantees each round contributes **exactly one** additional edge to every path under consideration.

Algorithm

1. Initialize $dist[src] = 0$, all others ∞ .
2. Repeat $k+1$ times: copy *dist* into *temp*; for every flight (u, v, w) , if $dist[u] + w < temp[v]$, update $temp[v]$; then set $dist \leftarrow temp$.
3. Return $dist[dst]$ if finite, else -1 .

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int findCheapestPrice(int n, vector<vector<int>>& flights, int src, int dst, int k)
5 {
6     vector<int> dist(n, INT_MAX);
7     dist[src] = 0;
8
9     for (int round = 0; round <= k; round++) {
10         vector<int> temp = dist;
11         for (auto& f : flights) {
12             int u = f[0], v = f[1], w = f[2];
13             if (dist[u] != INT_MAX && dist[u] + w < temp[v]) {
14                 temp[v] = dist[u] + w;
15             }
16         }
17         dist = temp;
18     }
19     return dist[dst] == INT_MAX ? -1 : dist[dst];
20 }
```

```

20
21 int main() {
22     vector<vector<int>> flights = {{0,1,100},{1,2,100},{0,2,500}};
23     cout << findCheapestPrice(3, flights, 0, 2, 1) << endl; // 200
24     return 0;
25 }

```

Dry run

Setup. Take $flights = [[0, 1, 100], [1, 2, 100], [0, 2, 500]]$, $src = 0$, $dst = 2$, $k = 1$ (at most 1 stop, i.e. at most 2 edges).

$dist = [0, \infty, \infty]$. Round 0 ($k+1 = 2$ rounds total): $temp = [0, \infty, \infty]$; relax $0 \rightarrow 1$: $0 + 100 = 100 < \infty$, $temp[1] = 100$; relax $1 \rightarrow 2$: $dist[1] = \infty$ still (reading the *frozen* pre-round value), skip; relax $0 \rightarrow 2$: $0 + 500 = 500 < \infty$, $temp[2] = 500$. $dist = [0, 100, 500]$. Round 1: $temp = [0, 100, 500]$; relax $0 \rightarrow 1$: 100 not < 100 , no change; relax $1 \rightarrow 2$: $dist[1] + 100 = 100 + 100 = 200 < 500$, $temp[2] = 200$; relax $0 \rightarrow 2$: 500 not < 200 , no change. $dist = [0, 100, 200]$. Final answer: $dist[2] = 200$ – the path $0 \rightarrow 1 \rightarrow 2$ (one stop, total 200), correctly preferred over the direct $0 \rightarrow 2$ edge costing 500.

Complexity

Time Complexity. $O(k \cdot E)$ – $k+1$ rounds, each scanning every flight once. **Space Complexity.** $O(V)$ for $dist$ and $temp$.

Takeaway

This problem is a useful warning against reaching for Dijkstra reflexively on every “cheapest path” phrasing: the moment a constraint couples the answer to *how the path is structured* (here, edge count) rather than purely to its total weight, Dijkstra’s greedy finalization can be provably wrong, and a bounded, round-based relaxation (Bellman-Ford’s core idea, capped early) becomes the correct tool.

1.7 Bellman-Ford Algorithm

Problem Statement. Given a weighted directed graph, possibly containing **negative** edge weights, find the shortest distance from a source to every other node – or detect that a **negative-weight cycle** makes the notion of “shortest path” undefined.

Pattern Recognition

“Shortest path, negative weights allowed” is solved by relaxing **every edge**, $V-1$ times over (no priority queue, no greedy frontier choice – Dijkstra’s greedy finalization is provably unsafe the moment a negative edge exists, as discussed in Section 1.3). A final, V -th round that still finds an improvement reveals a negative cycle.

Why $V - 1$ rounds are exactly enough

In a graph with no negative cycle, the shortest path between any two nodes is always a **simple** path (visits no node twice) – a path revisiting a node would include a cycle, and removing that cycle could only shorten the path further unless the cycle’s total weight is *exactly* 0, in which case removing it doesn’t change the length either way. A simple path in a graph with V nodes

has at most $V-1$ edges. Each full round of relaxing every edge extends the *confirmed-correct* prefix of every shortest path by (at least) one more edge – so after $V-1$ rounds, every shortest path (having at most $V-1$ edges) is guaranteed fully relaxed and correct.

Why a V -th round detects a negative cycle

If, after $V-1$ rounds (when every legitimate shortest path must already be finalized), a V -th round of relaxation *still* finds an improvement on some edge, that improvement cannot correspond to any genuine shortest path (those are already settled) – it can only be explained by a cycle whose total weight is negative, allowing “distance” to be driven arbitrarily low by looping through it repeatedly.

Algorithm

1. Initialize $\text{dist}[\text{src}] = 0$, all others ∞ .
2. Repeat $V-1$ times: for every edge (u, v, w) , if $\text{dist}[u] \neq \infty$ and $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v]$.
3. Run one more relaxation pass over all edges; if any edge can still be relaxed, report a negative cycle.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> bellmanFord(int n, vector<vector<int>>& edges, int src) {
5      vector<int> dist(n, INT_MAX);
6      dist[src] = 0;
7
8      for (int i = 0; i < n - 1; i++) {
9          for (auto& e : edges) {
10             int u = e[0], v = e[1], w = e[2];
11             if (dist[u] != INT_MAX && dist[u] + w < dist[v]) {
12                 dist[v] = dist[u] + w;
13             }
14         }
15     }
16
17     for (auto& e : edges) {
18         int u = e[0], v = e[1], w = e[2];
19         if (dist[u] != INT_MAX && dist[u] + w < dist[v]) {
20             return {-1}; // sentinel: negative cycle detected
21         }
22     }
23     return dist;
24 }
25
26 int main() {
27     vector<vector<int>> edges = {{0,1,4},{0,2,5},{1,2,-3},{2,3,4}};
28     vector<int> dist = bellmanFord(4, edges, 0);
29     for (int d : dist) cout << d << " ";
30     cout << endl; // 0 4 1 5
31     return 0;
32 }
```

Dry run

Setup. Take edges $0 \rightarrow 1(4)$, $0 \rightarrow 2(5)$, $1 \rightarrow 2(-3)$, $2 \rightarrow 3(4)$, $n = 4$, source 0.

$dist = [0, \infty, \infty, \infty]$. Round 1: relax $0 \rightarrow 1$ ($\infty \rightarrow 4$); relax $0 \rightarrow 2$ ($\infty \rightarrow 5$); relax $1 \rightarrow 2$ ($4 - 3 = 1 < 5$, update to 1); relax $2 \rightarrow 3$ ($1 + 4 = 5 < \infty$, update to 5). $dist = [0, 4, 1, 5]$. Round 2: relax $0 \rightarrow 1$ (4 not < 4); relax $0 \rightarrow 2$ (5 not < 1); relax $1 \rightarrow 2$ (1 not < 1); relax $2 \rightarrow 3$ (5 not < 5) – no changes, **stabilized** early (rounds can stop as soon as a full pass makes no change, though the algorithm above runs the full $V-1$ rounds regardless). Round 3 ($n-1 = 3$ total): no changes. Final check round: no edge relaxes further – no negative cycle. Final: $[0, 4, 1, 5]$.

Complexity

Time Complexity. $O(V \cdot E)$ – $V-1$ rounds (plus one check round), each scanning all E edges. **Space Complexity.** $O(V)$ for $dist$.

Takeaway

Bellman-Ford trades Dijkstra's $O((V+E) \log V)$ speed for two things Dijkstra cannot offer: correctness with negative edges, and the ability to detect negative cycles at all. When a problem's weights are guaranteed non-negative, Dijkstra remains the faster default – Bellman-Ford is the fallback specifically for when that guarantee is unavailable.

1.8 Floyd-Warshall Algorithm

Problem Statement. Given a weighted directed graph (as an $n \times n$ adjacency matrix, with ∞ for missing edges), find the shortest distance between **every pair** of nodes.

Pattern Recognition

“Shortest path between every pair of nodes, simultaneously” is solved not by running Section 1.3's Dijkstra from every node (which would cost $O(V \cdot (V+E) \log V)$), but by a direct dynamic programming recurrence over which nodes are **allowed as intermediate stops**: progressively allow node 1, then nodes 1 and 2, then 1, 2, 3, and so on, updating every pair's shortest distance each time a new node becomes available as a potential stop along the way.

Deriving the recurrence

Let $dist_k[i][j]$ be the shortest distance from i to j using **only** nodes $1, \dots, k$ as intermediate stops (besides i, j themselves). Either the shortest such path doesn't use node k at all (so $dist_k[i][j] = dist_{k-1}[i][j]$), or it does use k – exactly once, as a single stop, splitting the path into an $i \rightarrow k$ segment and a $k \rightarrow j$ segment, both of which only need nodes $1, \dots, k-1$ as their own intermediates (since they don't revisit k):

$$dist_k[i][j] = \min \left(dist_{k-1}[i][j], dist_{k-1}[i][k] + dist_{k-1}[k][j] \right).$$

Implemented in place, with k as the **outermost** loop, this needs only a single 2D array (each cell is read and written using already-updated values from the current and prior k , which is safe because $dist[i][k]$ and $dist[k][j]$ are never themselves updated using k as an intermediate – a path from i to k using k itself as a stop would be nonsensical).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int INF = 1e9;
4
5 void floydWarshall(vector<vector<int>>& dist) {
6     int n = dist.size();
7
8     for (int k = 0; k < n; k++) {
9         for (int i = 0; i < n; i++) {
10             for (int j = 0; j < n; j++) {
11                 if (dist[i][k] < INF && dist[k][j] < INF) {
12                     dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
13                 }
14             }
15         }
16     }
17 }
18
19 int main() {
20     int n = 4;
21     vector<vector<int>> dist(n, vector<int>(n, INF));
22     for (int i = 0; i < n; i++) dist[i][i] = 0;
23     dist[0][1] = 5; dist[1][2] = 3; dist[2][3] = 1; dist[0][3] = 100;
24
25     floydWarshall(dist);
26     cout << dist[0][3] << endl; // 9 (via 0->1->2->3: 5+3+1)
27     return 0;
28 }

```

Dry run

Setup. Take the 4-node graph above: $0 \rightarrow 1(5)$, $1 \rightarrow 2(3)$, $2 \rightarrow 3(1)$, $0 \rightarrow 3(100)$.

Initial $dist[0][3] = 100$ (direct edge). After allowing $k = 1$: $dist[0][2] = \min(\infty, dist[0][1] + dist[1][2]) = \min(\infty, 5 + 3) = 8$. After allowing $k = 2$: $dist[0][3] = \min(100, dist[0][2] + dist[2][3]) = \min(100, 8 + 1) = 9$. After allowing $k = 3$: no further improvement possible through 3 (it has no outgoing edges in this example). Final $dist[0][3] = 9$, matching the path $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$.

Complexity

Time Complexity. $O(V^3)$ – three nested loops over all nodes. **Space Complexity.** $O(V^2)$ for the distance matrix (updated in place; no extra space needed per layer of k).

Takeaway

Floyd-Warshall is this book’s clearest example of an **ending-here**-style DP (Section 1.1’s opening idea, generalized) applied to a 2D state (i, j) with a third dimension – “which nodes are allowed as intermediates” – collapsed into a single in-place array by processing k in the outer loop, the same in-place-update trick used by every space-optimized DP table throughout this book series.

1.9 City With the Smallest Number of Neighbors at a Threshold Distance

Problem Statement. Given n cities connected by weighted edges and a *distanceThreshold*, find the city from which the **fewest** other cities are reachable within *distanceThreshold*. If multiple cities tie for fewest, return the one with the **largest** index.

Pattern Recognition

This closing problem of the chapter is a direct application of Section 1.8: once **all-pairs** shortest distances are known, answering “how many cities are within threshold T of city i ” for every i is a simple linear scan over each row of the distance matrix – no further graph algorithm is needed.

Reduction

1. Build the $n \times n$ distance matrix and run Floyd-Warshall to fill in all-pairs shortest distances.
2. For each city i , count how many cities $j \neq i$ satisfy $\text{dist}[i][j] \leq \text{distanceThreshold}$.
3. Track the city with the smallest count, breaking ties by preferring the **larger** index (scan i from 0 to $n-1$, and use $\leq$ rather than $<$ when comparing counts, so a later equal-or-better city always overwrites an earlier one).

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  const int INF = 1e9;
4
5  int findTheCity(int n, vector<vector<int>>& edges, int distanceThreshold) {
6      vector<vector<int>> dist(n, vector<int>(n, INF));
7      for (int i = 0; i < n; i++) dist[i][i] = 0;
8      for (auto& e : edges) {
9          dist[e[0]][e[1]] = e[2];
10         dist[e[1]][e[0]] = e[2];
11     }
12
13     for (int k = 0; k < n; k++) {
14         for (int i = 0; i < n; i++) {
15             for (int j = 0; j < n; j++) {
16                 if (dist[i][k] < INF && dist[k][j] < INF) {
17                     dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
18                 }
19             }
20         }
21     }
22
23     int bestCity = -1, minCount = n;
24     for (int i = 0; i < n; i++) {
25         int count = 0;
26         for (int j = 0; j < n; j++) {
27             if (i != j && dist[i][j] <= distanceThreshold) count++;
28         }
29         if (count <= minCount) { // <= prefers the larger index on ties
30             minCount = count;

```

```

31         bestCity = i;
32     }
33 }
34 return bestCity;
35 }
36
37 int main() {
38     vector<vector<int>> edges = {{0,1,3},{1,2,1},{1,3,4},{2,3,1}};
39     cout << findTheCity(4, edges, 4) << endl; // 3
40     return 0;
41 }

```

Dry run

Setup. Take $n = 4$, edges $0-1(3), 1-2(1), 1-3(4), 2-3(1)$, $distanceThreshold = 4$.

All-pairs distances after Floyd-Warshall: from 0: $[0, 3, 4, 5]$ (to 2: $3 + 1 = 4$ via 1; to 3: $3 + 1 + 1 = 5$ via 1, 2, shorter than the direct $1-3$ edge route $3 + 4 = 7$). From 1: $[3, 0, 1, 2]$. From 2: $[4, 1, 0, 1]$. From 3: $[5, 2, 1, 0]$ (to 1: $1 + 1 = 2$ via 2, shorter than the direct edge 4). Counts within threshold 4 (excluding self): city 0 sees 1 (dist 3) and 2 (dist 4) – 2 cities (3 at distance 5 is excluded). City 1 sees all three others – 3 cities. City 2 sees all three others – 3 cities. City 3 sees 1 (dist 2) and 2 (dist 1) – 2 cities (0 at distance 5 is excluded). Cities 0 and 3 tie at 2 neighbors each; the tie-break (larger index) selects city 3.

Complexity

Time Complexity. $O(V^3)$ for Floyd-Warshall, plus $O(V^2)$ for the counting scan: $O(V^3)$ overall. **Space Complexity.** $O(V^2)$ for the distance matrix.

Takeaway

This chapter closes by demonstrating exactly why Floyd-Warshall earns its $O(V^3)$ cost despite being asymptotically worse than running Dijkstra from every node ($O(V(V + E) \log V)$ for sparse graphs): once *every* pairwise distance is available at once, follow-up questions like this one (counting neighbors within a threshold, for every city) become trivial linear scans – exactly the same trade-off this book has made repeatedly between an upfront, fuller computation and cheap, flexible queries afterward.